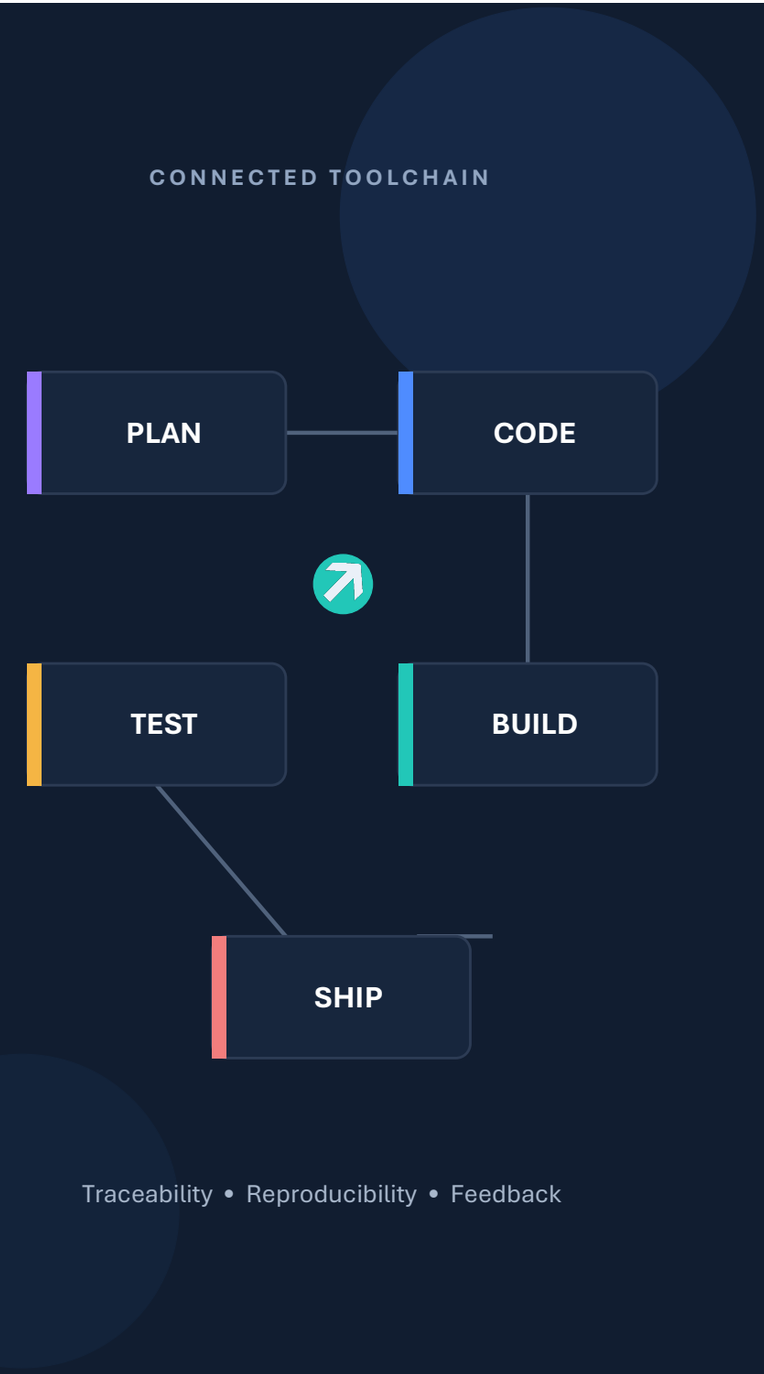
PRELIMINARY REPORT / 2026

Best Tools for Program Development

An Analysis of Modern Software Engineering Toolchains & Project Traceability

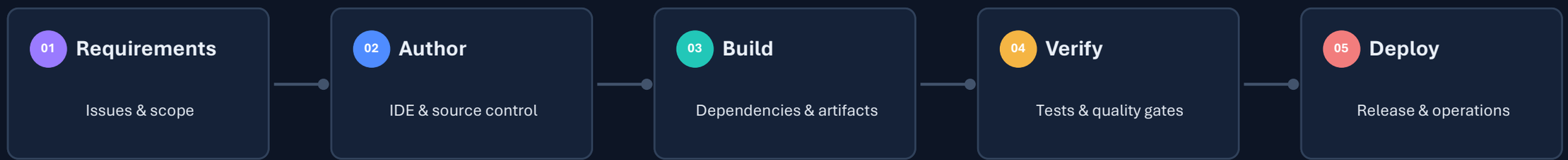
Șerban Alexandru-Cristian-George

Tools for Program Development – Preliminary Report



Introduction & SDLC Foundations

A modern toolchain connects people, code, automation, and evidence across the software lifecycle.



THE SHIFT

From isolated coding to pipelines

Teams rely on coordinated workflows instead of stand-alone editors and manual handoffs.

WHY IT MATTERS

Quality, speed, maintainability

Tool choices shape feedback time, consistency, collaboration, and long-term change cost.

OBJECTIVE

One cohesive, reproducible system

Connect requirements to code, tests, and releases so project progress can be verified.

Historical Evolution of Development Tools

Each era adds abstraction and automation while preserving the value of clear, inspectable workflows.

1970s

Command line & Unix

- Modular CLI utilities: vi, ed, cc
- make (1976) automates dependency-aware builds

Composable tools

1980s–1990s

Integrated environments

- Editors, compilers & debuggers brought together
- RCS / CVS and later SVN use a centralized model

Integrated workflow

2000s–2010s

Open source & containers

- SVN/CVS → Git (2005): every clone stores the full repository history
- Docker (2013) makes environments portable

Full local history

2020s–present

Cloud-native & AI-assisted

- Declarative CI/CD and LSP-powered tooling
- AI-assisted coding joins automated workflows

Connected automation

From local commands → integrated tools → distributed collaboration → continuously verified delivery

Core Tool Categories · Part 1

Choose an authoring environment that fits the language and workflow; use version control as the shared history.

01 / CODE AUTHORING

Integrated Development Environments

Visual Studio Code

LIGHTWEIGHT + MODULAR

LSP decouples the editor from language servers for completion, diagnostics, and navigation.

JetBrains ecosystem

DEEP LANGUAGE INTELLIGENCE

IntelliJ, CLion, and Rider emphasize semantic analysis and refactoring.

Visual Studio

ENTERPRISE-GRADE IDE

Integrated debugging for C# / .NET and substantial C++ solutions.

02 / VERSION CONTROL

Git keeps the history distributed

- **Local branches** — Parallel work
- **Integrity** — Content hashes
- **Audit trail** — Reviewable change

HOSTING + COLLABORATION

GitHub

Pull requests
Issues & Actions

GitLab

Merge requests
Issues & GitLab CI

Best practice: review changes before merging to the main branch.

Core Tool Categories · Part 2

Build tools make outputs repeatable; containers reduce environment drift; QA tools shorten the feedback loop.

01 Build & packages

Repeatable dependency graphs

C / C++
CMake

JVM
Gradle · Maven

PACKAGES
npm · Cargo · NuGet

Pin and lock dependencies

02 Containers & CI/CD

Consistent environments, automated flow

ENVIRONMENT
Docker · Podman

CI PLATFORMS
GitHub Actions

PIPELINE AUTOMATION
GitLab CI

Build → test → deliver

03 Testing & QA

Catch regressions and risky patterns

UNIT TESTS
xUnit · pytest

INTEGRATION / API
Postman

STATIC + SECURITY GATES
SonarQube

Unit → API → security gates

Toolchain Comparison Matrix

The strongest stack is the one that covers the lifecycle with minimal friction and visible evidence.

CATEGORY	LEADING TOOLS	CORE STRENGTH	SDLC STAGE
Code authoring	VS Code JetBrains Visual Studio	Smart completion, refactoring, debugging	Implementation
Version control	Git GitHub GitLab	Branching, review workflows, audit trail	All SDLC phases
Build & dependencies	CMake Gradle Cargo npm	Reproducible builds and dependency control	Build
Environment parity	Docker Podman	Process isolation; consistent dev/test/prod	Testing & deployment
Automated CI/CD	GitHub Actions GitLab CI	Automated checks and rapid feedback	Verification & delivery
Testing & quality	Postman SonarQube xUnit pytest	Regression detection, API checks, code health	Quality assurance

Traceability in Practice: GitHub Workflow

Follow one ML classification feature from issue definition to a versioned release.

ILLUSTRATIVE ISSUE-TO-RELEASE WORKFLOW



Commit convention: feat(ml): add ticket priority prediction (refs #12)

01 Interoperability

CLI, webhooks, and APIs should connect the tools without manual copying.

02 Reproducibility

Lock dependencies and standardize environments so runs behave consistently.

03 Velocity & feedback

Fast tests, linting, and quality gates catch defects close to the change.

Course brief: project traceability is explicitly emphasized in the final score; GitHub or a similar VCS is recommended.

Our Proposed Project Stack

Selected Baseline for the Semester Project — a practical starting point for Report 1.

GitHub

VCS + ISSUE TRACKER

Issues link the feature request to branches, pull requests, workflow runs, and release evidence.

Issue #12 → PR #15

VS Code / Visual Studio

AUTHORING + DEBUGGING

Use LSP-aware editing and a language-specific debugger matched to the project's language and framework.

feature/ml-ticket-prediction

Docker

REPRODUCIBLE ENVIRONMENT

Pin the runtime and dependencies so local development and CI use a consistent environment.

Dockerfile

GitHub Actions

CONTINUOUS INTEGRATION

Run the build, tests, and linter automatically on push and pull_request events.

.github/workflows/ci.yml

TRACEABILITY TRAIL

Issue #12 → feat(ml): add ticket priority prediction (refs #12)→ PR #15 → green CI run → tag v1.0.0

Baseline principle: integrated tools, reproducible runs, and evidence that can be inspected during grading.

Conclusions & References

The best tools work as an integrated system: each reduces friction and creates evidence for the next stage.

KEY TAKEAWAYS

01 **Think ecosystem, not product**

A connected toolchain is more valuable than isolated tools used without shared workflow.

02 **Start with version control**

Adopt Git and a hosted collaboration workflow from day one to preserve change history.

03 **Automate the evidence**

Repeatable builds, tests, reviews, and releases help demonstrate quality and accountability.

REFERENCES

01 **Software Testing Help**

“Top 20+ Best Software Development Tools”

softwaretestinghelp.com/software-development-tools/

02 **Guru99**

“24 Best Software Development Tools Ranking”

guru99.com/software-development-tools.html

03 **Chacon, S. & Straub, B.**

Pro Git. Apress.

git-scm.com/book

04 **Sommerville, I.**

Software Engineering, 10th Edition. Pearson.

Pearson Education